

Volume 05

User Experience & Accessibility

Chapter 1

Human-Centered Design

Project Sentinel shall be designed for humans first.

The platform shall support:

- beginners
- professionals
- administrators
- security analysts
- developers
- auditors
- government users

without requiring different applications.

Chapter 2

Accessibility Principles

The platform shall follow:

- Perceivable
- Operable
- Understandable
- Robust

These are the four WCAG principles.

Chapter 3

Keyboard Navigation

Every function shall be usable without a mouse.

Examples:

Tab navigation

Logical focus order

Skip navigation links

Keyboard shortcuts

Visible focus indicator

No critical functionality may require a pointing device.

Chapter 4

Screen Reader Support

The interface shall support screen readers by:

- semantic HTML
 - proper headings
 - landmarks
 - labels
 - accessible forms
 - descriptive button names
 - meaningful link text
 - ARIA only where appropriate (native HTML is preferred whenever possible)
-

Chapter 5

Color Independence

No information shall rely only on color.

Example:

Bad

Critical

Good

Critical

High Risk

The icon and text remain understandable even without color perception.

Chapter 6

Contrast

The interface shall maintain sufficient contrast according to WCAG 2.2 AA requirements.

This applies to:

- text
 - buttons
 - icons
 - charts
 - tables
 - dialogs
-

Chapter 7

Responsive Design

Project Sentinel shall support:

Desktop

Laptop

Tablet

Mobile

Large displays

without loss of functionality.

Chapter 8

Language Support

The architecture shall support localization.

Initial languages:

- English
- Norwegian

Additional languages can be installed as language packs.

All user-visible strings shall be externalized to translation resources rather than hard-coded.

Chapter 9

Dark Mode

The platform shall support:

Light Mode

Dark Mode

High Contrast Mode

System Default

The user's preference shall be remembered.

Chapter 10

Readability

The platform shall use:

- consistent typography
 - predictable layouts
 - plain language where appropriate
 - technical terminology only when necessary
-

Chapter 11

Progressive Disclosure

Advanced options shall remain hidden until requested.

Example

Normal User

Scan Website

Expert User

Advanced Scan Profile

Same platform.

Different complexity.

Chapter 12

Error Messages

Error messages shall:

Explain

Why

What happened

How to fix it

Example

Instead of

HTTP Error 403

The platform explains

Authentication failed.

Chapter 13

Accessibility Testing

Every release shall include:

- automated accessibility testing
- manual keyboard testing
- screen reader verification
- contrast verification
- responsive verification

Accessibility becomes part of the release criteria.

New Engineering Rule

I want to introduce something that I have not seen consistently implemented in security products.

Cognitive Load Budget

Every screen shall have a maximum cognitive complexity.

This means we measure and review:

- Number of primary actions
- Number of visible controls
- Information density
- Required decisions
- Time to complete common tasks

If a screen becomes too complex, it must be redesigned before release.

This supports our principle:

Power without Complexity

Another Major Proposal

I also want to introduce an **Architecture Compliance Matrix (ACM)**.

Every module must demonstrate compliance with the platform standards before it is accepted.

Example:

Requirement	Status
-------------	--------

Security by Design	
--------------------	--

One important observation

As we continue, I also want us to **separate mandatory requirements from optional enhancements**.

For example:

Mandatory (Core Requirements):

- WCAG 2.2 AA compliance
- Security by Design
- Privacy by Design
- Explainability
- RBAC
- Audit logging

- Offline capability
- Internationalization
- API documentation
- Automated testing

Optional (Future Enhancements):

- Experimental AI features
- Additional visualization styles
- New plugin categories
- Advanced research capabilities

That distinction will keep Project Sentinel focused, achievable, and easier to deliver while still leaving plenty of room for innovation in future releases. I believe this discipline will be one of the reasons the platform succeeds as an engineering project rather than becoming an ever-growing collection of ideas.

====[USER]=====

And as we discuss before, easy to use and easy to scan also mobil device and fix the issue if found. Good and continue

====[ASSISTANT]=====

Excellent. I also want to refine one important point to keep the project realistic and legally sound.

Earlier we discussed that Project Sentinel should be able to **"fix the issue if found."** I think we should define this more precisely.

The platform should support **three remediation modes**:

1. **Recommend** (default)

- Explain the issue.
- Explain the risk.
- Show the recommended fix.
- Generate scripts or configuration examples where appropriate.

2. **Guided Remediation**

- The user explicitly approves an action.
- Project Sentinel performs the approved action.
- Every action is logged and reversible where possible.

3. **Automatic Remediation**

- Only for predefined, low-risk operations that the administrator has explicitly enabled.
- Examples:
 - Renew a certificate.
 - Disable an inactive account.
 - Rotate an API key.
 - Update firewall rules from an approved policy.
- Potentially disruptive actions (such as applying operating system patches or deleting software) should always require explicit approval or organizational policy.

This keeps the platform safe and suitable for enterprise and government use.

PROJECT SENTINEL BLUEPRINT

Volume 06